



南京林业大学
NANJING FORESTRY UNIVERSITY

FeHALS 航路规划与激光仿真系统 API 技术文档

课程名称：GIS 设计与开发

专业：测绘工程

组别：7 组

年级：2023 级

小组成员：	王孜悠	2310604116
	梅康凯	2310604109
	唐如意	2310604115
	杨杰瑞	2310604122
	温璞	2310604117
	刘继明	2110604112
	李英杰	2110604106
	杜仲宇	2310604103
	施祺	2310604113
	向宇鸣	2310604118

指导老师：隋铭明 那嘉明 陈动 朱杰

批改日期：

提交日期：2026 年 9 月 4 日

目录

- 1 系统概述 3
 - 1.1 项目背景 3
 - 1.2 系统架构 3
- 2 相关技术 3
 - 2.1 HELIOS++ 仿真框架 3
 - 2.2 Web 三维可视化与 Three.js 4
 - 2.3 Vue 3 前端框架 4
 - 2.4 FastAPI 与后端异步任务 4
- 3 系统需求分析与设计 4
 - 3.1 用户需求分析 4
 - 3.2 系统总体架构 4
 - 3.3 功能优先级划分 4
- 4 系统设计与实现 5
 - 4.1 系统模块划分 5
 - 4.2 三维场景渲染与航点交互 5
 - 4.3 前端组件与状态管理 5
 - 4.4 后端 API 接口设计 5
 - 4.5 航迹与配置文件生成 5
 - 4.6 HELIOS++ 调用与点云渲染 6
 - 4.7 项目组织结构 6
- 5 部署与运行指南 6
 - 5.1 环境要求 6
 - 5.2 安装步骤 6
 - 5.3 启动脚本 8
 - 5.4 配置说明 9
- 6 API 接口详解 9
 - 6.1 REST API 接口 9
 - 6.1.1 模型管理接口 9
 - 6.1.2 航迹管理接口 10
 - 6.1.3 仿真执行接口 11
 - 6.1.4 结果管理接口 12
 - 6.2 WebSocket 接口 12
 - 6.2.1 日志推送 12
 - 6.3 数据模型定义 13

6.3.1	航点数据模型	13
6.3.2	仿真参数数据模型	14
6.3.3	任务状态数据模型	14
7	前端实现详解	15
7.1	Vue 3 应用架构	15
7.1.1	组件结构	15
7.1.2	3D 场景组件实现	15
7.2	Three.js 集成与 3D 交互	17
7.2.1	模型加载系统	17
7.2.2	航点交互系统	19
8	后端实现详解	22
8.1	FastAPI 应用架构	22
8.1.1	主应用配置	22
8.1.2	配置管理系统	23
8.2	API 路由实现	24
8.2.1	REST 路由	24
8.3	业务服务层	25
8.3.1	航迹生成服务	25
8.3.2	点云处理服务	28
A	部署与配置管理	33
A.1	环境变量配置	33
A.2	Docker 部署方案	33
A.3	服务监控与日志	34
B	开发规范与最佳实践	34
B.1	代码规范	34
B.2	API 设计原则	34
C	故障排查指南	34

1. 系统概述

1.1. 项目背景

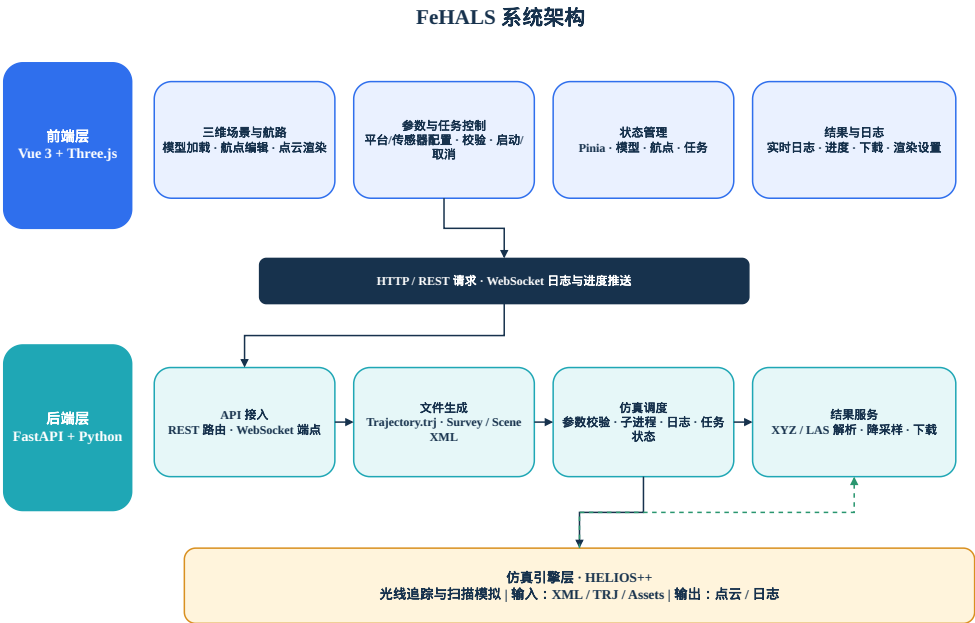
FeHALS (Frontend of HELIOS++ for Airborne Laser Scanning) 是一个基于 Web 的三维可视化航路规划与激光仿真系统，旨在解决 HELIOS++ 原生命令行交互复杂、缺乏三维可视化航路规划能力的问题。本系统为航测工程师、遥感研究人员以及相关专业师生提供一个直观易用的平台，支持从场景构建、航路设计、仿真执行到结果可视化的全流程操作。

1.2. 系统架构

系统采用浏览器/服务器 (B/S) 三层架构：

- 前端层：基于 Vue 3 与 Three.js 构建，负责三维场景渲染、航点交互编辑、参数配置与结果展示
- 后端层：基于 FastAPI 实现，负责 API 服务、文件生成、仿真调度与数据处理
- 仿真引擎层：HELIOS++ 激光扫描仿真引擎，独立运行并由后端调用

Fig. 1: FeHALS 系统架构与数据流



2. 相关技术

2.1. HELIOS++ 仿真框架

HELIOS++ 是一款基于 C++ 的开源激光雷达仿真框架，采用光线追踪技术对激光脉冲在三维场景中的传播过程进行建模，能够模拟机载、地面、车载与无人机等多种平台，支持全波形 (full-waveform) 输出与 LAS、LAZ、XYZ 等多种点云格式。其仿真配置以 XML 文件组织：场景 (scene) 文件定义参与扫描的三维地物模型；平台 (platform) 与扫描器 (scanner) 目录文件定义飞行平台与扫描仪的物理

参数；扫描任务（survey）文件将场景、平台、扫描器与航迹（trajectory）关联起来，并设定脉冲频率、扫描频率与扫描角等核心参数。

2.2. Web 三维可视化与 Three.js

Three.js 是一个基于 WebGL^[1] 的开源 JavaScript 三维图形库，封装了场景（Scene）、相机（Camera）、渲染器（Renderer）与网格（Mesh）等核心对象，支持 OBJ、GLTF、STL 等多种三维模型格式的加载，并提供了轨道控制（OrbitControls）、拖拽控制（DragControls）与射线拾取（Raycaster）等交互组件。借助 Three.js，可在浏览器中构建轻量级的三维可视化场景，实现模型展示、点云渲染与鼠标交互拾取，为航路规划提供直观的操作界面。

2.3. Vue 3 前端框架

Vue 3 是一个渐进式 JavaScript 框架，采用 Composition API 提供更好的代码组织和类型支持。其核心特性包括响应式系统、组件化开发和虚拟 DOM。本系统前端基于 Vue 3 构建，使用 Pinia 进行状态管理，通过组件化方式组织用户界面，实现了高效的 3D 交互和参数配置功能。

2.4. FastAPI 与后端异步任务

FastAPI 是一个基于 Python 的高性能 Web 框架，基于类型注解与 Pydantic^[2] 实现自动的数据校验与接口文档生成，并原生支持异步编程与 WebSocket。本系统后端基于 FastAPI 构建 REST API^[3] 与 WebSocket 日志通道，通过 asyncio 子进程异步调用 HELIOS++ 可执行文件，实时捕获其标准输出并以 WebSocket 推送到前端，实现仿真过程的实时监控。

3. 系统需求分析与设计

3.1. 用户需求分析

本系统的目标用户为航测工程师、遥感研究人员以及相关专业师生。其核心需求可归纳为三个方面：一是直观的航路规划，用户希望在不编写配置文件的情况下，通过三维场景中的鼠标交互即可完成航点设定与航线编辑；二是快速的仿真执行，用户能够通过图形化界面配置平台与扫描仪参数，一键调用 HELIOS++ 引擎并实时查看运行状态；三是便捷的结果可视化，仿真生成的点云能够自动加载到三维场景中，并支持按高度或强度着色以及尺寸、透明度的调节。

3.2. 系统总体架构

系统采用浏览器/服务器（B/S）架构，分为前端、后端与仿真引擎三层，其总体架构如图1所示。前端基于 Vue 3 构建，负责三维场景渲染、航点交互编辑、参数配置与结果展示，通过 HTTP^[4-6] 与 WebSocket 与后端通信；后端基于 FastAPI，负责接收前端请求，生成 HELIOS++ 所需的航迹文件与 XML 配置文件，调用 HELIOS++ 可执行文件执行仿真，并解析与返回点云结果；仿真引擎层为 HELIOS++，独立于前后端，由后端以子进程方式调用。各模块间通过明确的 REST API 与 WebSocket 消息格式进行数据交换，形成完整的工作流。

3.3. 功能优先级划分

结合开发环境与工程约束，系统功能划分为三个优先级：核心功能（MVP）包括三维场景渲染、模型加载、交互式航点编辑、航迹导出、仿真参数配置、仿真执行、日志控制台与点云渲染；重要功能包

括场景要素属性编辑、自动航线生成、航高自动计算、仿真预设方案、点云特征统计与点云下载；拓展功能包括仿真进度实时反馈、点云覆盖度分析、多任务队列管理与仿真过程动画。

4. 系统设计与实现

4.1. 系统模块划分

系统实现按前后端分层组织，前端基于 Vue 3 与 Node.js^[7] 生态构建，目录下包含组件（components）、状态管理（stores）与组合式函数（composables）三部分：组件层包含三维场景 Scene3D.vue、参数配置面板 ControlPanel.vue、航点列表 WaypointList.vue 与日志控制台 LogConsole.vue；stores 层基于 Pinia 维护场景（scene）、航点（waypoints）与仿真（simulation）三份响应式状态；composables 层封装 Three.js 场景管理（useThreeScene）、航点交互（useWaypoints）与后端接口调用（useHeliosAPI）。

后端基于 FastAPI 构建，api 目录下包含 REST 路由 routes.py 与 WebSocket 端点 websocket.py，models 目录定义 Pydantic 数据模型 schemas.py，services 目录封装航迹生成 trajectory_generator.py、配置生成 config_generator.py、HELIOS++ 调用 helios_service.py 与点云解析 pointcloud_parser.py。

4.2. 三维场景渲染与航点交互

三维场景基于 Three.js 构建，Scene3D.vue 负责初始化场景、相机、渲染器与灯光，并添加地面网格与拾取平面。场景使用 Z-up 坐标系（与 HELIOS++ 一致，z 为高程轴），通过 OrbitControls 实现相机的旋转、平移与缩放；模型加载采用 OBJLoader、GLTFLoader 与 STLLoader，加载完成后根据模型包围盒自动调整相机视角，并基于模型顶点包围盒推断其 up 轴（Y-up 模型自动旋转至 Z-up）。航点交互通过自绘拖拽实现：鼠标左键点击经 Raycaster 与地面平面求交，在交点处添加球体表示的航点（恒为 z=0 贴地），相邻航点之间以白色折线连接；拖拽航点时锁定相机（‘controls.enabled=false’），确保不干扰视角操作；通过 Delete 键或右键菜单删除航点。

4.3. 前端组件与状态管理

侧边栏采用多 Tab 布局，包含仿真参数、点云、模型列表、航迹与设置五个 Tab。ControlPanel.vue 提供仿真参数表单，分为「载体参数」与「传感器参数」两个模块，各参数旁显示有效范围，只读参数以灰色禁用显示；参数数据来源于从 HELIOS++ 扫描器与平台目录文件中提取的规格库，切换平台类型时自动重置参数至默认值。WaypointList.vue 以列表形式展示航点序号与坐标，支持单个删除与整体清空。LogConsole.vue 展示分级日志（INFO/WARNING/ERROR），支持自动滚动与清空。

三个 Pinia store 分别维护场景模型与点云渲染参数（scene）、航点坐标数组（waypoints）以及仿真任务状态、进度、日志与结果（simulation）。前端通过 useHeliosAPI.js 中的 Axios 实例调用后端 REST 接口，并通过 connectLogWS 建立 WebSocket 连接以接收仿真日志。

4.4. 后端 API 接口设计

后端基于 FastAPI 实现，提供完整的 RESTful API 接口，主要分为以下几类：

4.5. 航迹与配置文件生成

trajectory_generator.py 将航点数组按添加顺序、以 1 秒时间间隔生成 HELIOS++ 原生航迹文件（.trj，CSV 格式，列顺序为时间、roll、pitch、yaw、x、y、z），所有航点统一为恒定飞行高度（仅

表 1
FeHALS REST API 接口列表

接口组	HTTP 方法	功能说明
模型管理	GET /api/models	获取所有上传的模型列表
	POST /api/models/upload	上传 3D 模型文件
	DELETE /api/models/{id}	删除指定模型
航迹管理	POST /api/trajectory/generate	从航点生成航迹文件
	GET /api/trajectories	获取所有航迹列表
	GET /api/trajectories/download/{id}	下载航迹文件
配置管理	POST /api/config/generate	生成仿真配置文件
仿真执行	POST /api/simulation/run	执行 HELIOS++ 仿真
	GET /api/simulation/status/{task_id}	获取仿真状态
	GET /api/simulation/logs/{task_id}	获取仿真日志
	POST /api/simulation/cancel/{task_id}	取消仿真任务
结果管理	GET /api/results/{task_id}	获取点云数据
	GET /api/results/{task_id}/download	下载原始结果
环境诊断	GET /api/env/diagnose	检查系统环境

用 x、y 定义水平路径)。系统还支持弓字形自动航迹生成：用户在三维场景中点击两个角点定义矩形区域，系统根据设定航线间距自动生成往返覆盖的全覆盖航线。

config_generator.py 负责生成 HELIOS++ 所需的两个 XML 文件：场景文件通过 objloader 滤镜引用三维模型，并指定模型的 up 轴 (Y-up 自动旋转至 Z-up)；扫描任务 (survey) 文件引用场景、平台与扫描器目录，并将平台类型映射为对应的平台目录项 (UAV 与 Airborne 均使用 ‘copter_linearpath’ 平台、‘riegl_vux-1uav’ 扫描器)，将脉冲频率 (kHz) 与扫描角 (± 半角) 换算为 HELIOS++ 所需的脉冲频率 (Hz) 与总扫描角。

4.6. HELIOS++ 调用与点云渲染

helios_service.py 维护仿真任务注册表，通过 asyncio.create_subprocess_exec 以子进程方式调用 helios++ 可执行文件，指定资源目录 (-assets)、输出目录 (-output) 与输出格式参数，实时捕获其标准输出/错误流并解析进度百分比，经 WebSocket 推送至前端，同时支持超时终止与任务取消。

仿真结束后，pointcloud_parser.py 利用 laspy^[8] 解析 LAS/LAZ 点云或按文本方式解析 XYZ 点云，提取坐标与强度、计算空间范围，并对超过百万级的点云进行降采样。前端将返回的点云坐标构建为 BufferGeometry 与 Points 对象，支持按高度 (蓝色至红色渐变)、按强度或固定颜色着色，并可实时调节点尺寸与透明度。

4.7. 项目组织结构

5. 部署与运行指南

5.1. 环境要求

5.2. 安装步骤

1. 后端安装

Fig. 2: FeHALS 项目目录结构

```

1 FeHALS/
2   backend/           # 后端代码
3     app/
4       main.py        # FastAPI应用入口
5       config.py      # 配置管理
6       api/           # API路由
7       models/        # 数据模型
8       services/      # 业务服务
9       static/        # 静态文件目录
10      requirements.txt # Python依赖
11      run.py          # 后端启动脚本
12 frontend/          # 前端代码
13   src/
14     components/      # Vue组件
15     composables/     # 组合式函数
16     stores/          # Pinia状态管理
17     assets/          # 静态资源
18   package.json
19   vite.config.js
20   doc/              # 文档目录
21     src/
22       DESCRIPTION.tex
23       DESIGNBOOK.tex
24       DOCUMENT.tex
25   Makefile          # 构建脚本
26   run.sh            # Unix启动脚本

```

表 2

系统环境要求

组件	要求
Node.js	>= 16.0.0
Python	>= 3.8.0
npm	>= 7.0.0
LaTeX	XeLaTeX (用于文档编译)

```

1 # 1. 进入后端目录
2 cd backend
3
4 # 2. 创建虚拟环境
5 python -m venv venv
6
7 # 3. 激活虚拟环境
8 # Linux/Mac
9 source venv/bin/activate
10 # Windows
11 venv\Scripts\activate

```



```
12
13 # 4. 安装依赖
14 pip install -r requirements.txt
15
16 # 5. 设置环境变量
17 export HELIOS_PATH=/path/to/helios++
18 export HELIOS_REPO=/path/to/helios++/repo
19 export HELIOS_ASSETS=/path/to/assets
```

2. 前端安装

```
1 # 1. 进入前端目录
2 cd frontend
3
4 # 2. 安装依赖
5 npm install
6
7 # 3. 启动开发服务器
8 npm run dev
```

5.3. 启动脚本

项目提供了启动脚本用于快速启动服务：

```
1 #!/bin/bash
2
3 # 停止所有服务
4 stop_services() {
5     echo "停止所有服务..."
6     pkill -f "uvicorn" || true
7     pkill -f "vite" || true
8 }
9
10 # 启动后端
11 start_backend() {
12     echo "启动后端服务..."
13     cd backend
14     source venv/bin/activate
15     python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000 &
16     cd ..
17 }
18
19 # 启动前端
20 start_frontend() {
21     echo "启动前端服务..."
22     cd frontend
23     npm run dev &
24     cd ..
25 }
26
27 # 主程序
28 case "$1" in
29     start)
30         start_backend
```

```

31     start_frontend
32     echo "服务已启动"
33     echo "前端:␣http://localhost:3000"
34     echo "后端:␣http://localhost:8000"
35     echo "API文档:␣http://localhost:8000/docs"
36     ;;
37 stop)
38     stop_services
39     echo "服务已停止"
40     ;;
41 *)
42     echo "用法:␣$0␣{start|stop}"
43     exit 1
44     ;;
45 esac

```

Listing 1: run.sh 脚本

5.4. 配置说明

• 后端配置

```

1  # 环境变量示例
2  export FEHALS_HOST=0.0.0.0
3  export FEHALS_PORT=8000
4  export HELIOS_PATH=/usr/local/bin/helios++
5  export HELIOS_REPO=/opt/helios++
6  export HELIOS_ASSETS=/opt/helios++/assets
7  export FEHALS_SIM_TIMEOUT=300
8  export FEHALS_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000

```

• 前端配置

```

1  // vite.config.js
2  export default defineConfig({
3    server: {
4      proxy: {
5        '/api': {
6          target: 'http://localhost:8000',
7          changeOrigin: true
8        },
9        '/ws': {
10         target: 'ws://localhost:8000',
11         ws: true
12       }
13     }
14   }
15 })

```

6. API 接口详解

6.1. REST API 接口

6.1.1. 模型管理接口

表 3
模型管理 API 接口

接口路径	HTTP 方法	功能说明
/api/models	GET	获取所有已上传的模型列表
/api/models/upload	POST	上传 3D 模型文件（支持 OBJ、GLTF、STL 格式）
/api/models/{id}	DELETE	删除指定 ID 的模型
/api/models/{id}/info	GET	获取指定模型的详细信息
/api/models/{id}/bbox	GET	获取指定模型的包围盒信息

上传模型接口示例

```
1 # 请求示例（使用curl）
2 curl -X POST \
3   http://localhost:8000/api/models/upload \
4   -H "Content-Type: multipart/form-data" \
5   -F "file=@path/to/model.obj" \
6   -F "name=test_model" \
7   -F "description=测试模型文件"
8
9 # 响应示例（JSON）
10 {
11   "id": "model_123",
12   "name": "test_model",
13   "description": "测试模型文件",
14   "filename": "model.obj",
15   "size": 1024000,
16   "format": "obj",
17   "upload_time": "2024-01-01T12:00:00Z",
18   "bbox": {
19     "min": {"x": -10, "y": -10, "z": 0},
20     "max": {"x": 10, "y": 10, "z": 20}
21   }
22 }
```

Listing 2: 上传 3D 模型请求

6.1.2. 航迹管理接口

表 4
航迹管理 API 接口

接口路径	HTTP 方法	功能说明
/api/trajectory/generate	POST	从航点列表生成 HELIOS++ 航迹文件
/api/trajectories	GET	获取所有保存的航迹列表
/api/trajectories/{id}	GET	获取指定航迹的详细信息
/api/trajectories/{id}/download	GET	下载指定航迹的.trj 文件
/api/trajectories/{id}/preview	GET	获取航迹的 3D 预览数据

```
生成航迹接口示例
1 # 请求体 (JSON)
2 {
3   "waypoints": [
4     {"id": 1, "x": 0, "y": 0, "z": 100},
5     {"id": 2, "x": 100, "y": 0, "z": 100},
6     {"id": 3, "x": 100, "y": 100, "z": 100},
7     {"id": 4, "x": 0, "y": 100, "z": 100}
8   ],
9   "flight_height": 100,
10  "time_interval": 1.0,
11  "pattern": "manual"
12 }
13
14 # 生成的航迹文件格式 (CSV 示例)
15 time,roll,pitch,yaw,x,y,z
16 0.0,0.0,0.0,0.0,0.0,0.0,100.0
17 1.0,0.0,0.0,0.0,100.0,0.0,100.0
18 2.0,0.0,0.0,0.0,100.0,100.0,100.0
19 3.0,0.0,0.0,0.0,0.0,100.0,100.0
```

Listing 3: 生成航迹请求

6.1.3. 仿真执行接口

表 5
仿真执行 API 接口

接口路径	HTTP 方法	功能说明
/api/simulation/run	POST	启动 HELIOS++ 仿真任务
/api/simulation/status/{task_id}	GET	获取仿真任务状态
/api/simulation/logs/{task_id}	GET	获取仿真任务日志
/api/simulation/cancel/{task_id}	POST	取消正在执行的仿真任务
/api/simulation/restart/{task_id}	POST	重启失败的仿真任务
/api/simulation/history	GET	获取历史仿真任务列表

```
启动仿真接口示例
1 # 请求体 (JSON)
2 {
3   "survey_config": {
4     "platform": "copter_linearpath",
5     "scanner": "riegl_vux-1uav",
6     "trajectory_id": "trajectory_123",
7     "scene_id": "scene_456"
8   },
9   "simulation_params": {
10     "pulse_frequency": 100000,
11     "scan_frequency": 100,
12     "scan_angle": 15,
13     "flight_speed": 30,
14     "output_format": "LAS"
15   }
16 }
```

```
15 },
16 "output_settings": {
17   "output_path": "/tmp/simulation_results",
18   "file_prefix": "simulation_001",
19   "include_intensity": true,
20   "include_fullwave": false
21 }
22 }
23
24 # 任务状态响应
25 {
26   "task_id": "sim_789",
27   "status": "running",
28   "progress": 0.45,
29   "start_time": "2024-01-01T12:00:00Z",
30   "estimated_completion": "2024-01-01T12:05:00Z"
31 }
```

Listing 4: 启动仿真请求

6.1.4. 结果管理接口

表 6
结果管理 API 接口

接口路径	HTTP 方法功能说明	
/api/results/{task_id}	GET	获取仿真结果概览信息
/api/results/{task_id}/download	GET	下载完整的仿真结果文件
/api/results/{task_id}/stats	GET	获取点云统计数据
/api/results/{task_id}/preview	GET	获取点云预览数据（前 1000 点）
/api/results/{task_id}/downsample	POST	生成降采样后的点云数据
/api/results/{task_id}/export	POST	导出为指定格式的点云文件

6.2. WebSocket 接口

6.2.1. 日志推送

WebSocket 连接示例

```
1 // 建立WebSocket连接
2 const ws = new WebSocket('ws://localhost:8000/ws/simulation/sim_789');
3
4 // 接收日志消息
5 ws.onmessage = function(event) {
6   const data = JSON.parse(event.data);
7
8   switch(data.type) {
9     case 'log':
10       console.log(`[${data.level}] ${data.timestamp}: ${data.message}`);
11       updateProgress(data.progress);
12       break;
13     case 'status':
14       console.log(`Task ${data.task_id} status: ${data.status}`);
```

表 7

WebSocket 日志消息格式

消息类型	数据字段	说明
log	timestamp	日志时间戳
	level	日志级别 (INFO/WARNING/ERROR)
	message	日志消息内容
	progress	进度百分比 (0-100)
status	task_id	任务 ID
	status	任务状态 (running/completed/failed/cancelled)
	progress	当前进度
	error	错误信息 (如果有)
result	task_id	任务 ID
	result_path	结果文件路径
	point_count	点云总数
	file_size	结果文件大小

```

15         break;
16     case 'result':
17         console.log(`Simulation completed: ${data.result_path}`);
18         downloadResults(data.task_id);
19         break;
20     }
21 };
22
23 // 发送控制消息
24 function cancelSimulation() {
25     ws.send(JSON.stringify({
26         type: 'command',
27         action: 'cancel'
28     }));
29 }

```

Listing 5: WebSocket 客户端连接

6.3. 数据模型定义

6.3.1. 航点数据模型

```

1 # models/schemas.py
2 from pydantic import BaseModel
3 from typing import List, Optional
4 from datetime import datetime
5
6 class Waypoint(BaseModel):
7     id: int
8     x: float
9     y: float
10    z: float = 100.0

```

```

11     timestamp: Optional[datetime] = None
12
13 class WaypointList(BaseModel):
14     waypoints: List[Waypoint]
15     name: Optional[str] = "default_trajectory"
16     created_at: datetime = datetime.now()

```

Listing 6: Waypoint Pydantic 模型

6.3.2. 仿真参数数据模型

```

1 class PlatformConfig(BaseModel):
2     type: str # "copter_linearpath", "fixedwing", "ground"
3     name: str
4     speed: float
5     min_height: float
6     max_height: float
7
8 class ScannerConfig(BaseModel):
9     type: str # "riegl_vux-1uav", "livox-mid70", "custom"
10    pulse_frequency: int # Hz
11    scan_frequency: float # Hz
12    scan_angle: float # degrees
13    scan_direction: str # "nadir", "oblique"
14
15 class SimulationConfig(BaseModel):
16     platform: PlatformConfig
17     scanner: ScannerConfig
18     trajectory_id: str
19     output_format: str = "LAS"
20     output_path: str
21     timeout: int = 300 # seconds
22
23 # 全局配置常量 (实际应从config.py导入)
24 HELLOS_PATH = "/usr/local/bin/helios++"
25 HELLOS_REPO = "/opt/helios++/repo"
26 HELLOS_ASSETS = "/opt/helios++/assets"
27 MAX_FILE_SIZE = 100 * 1024 * 1024 # 100MB
28 ALLOWED_EXTENSIONS = [".obj", ".gltf", ".glb", ".stl"]

```

Listing 7: SimulationParams Pydantic 模型

6.3.3. 任务状态数据模型

```

1 class TaskStatus(str, Enum):
2     PENDING = "pending"
3     RUNNING = "running"
4     COMPLETED = "completed"
5     FAILED = "failed"
6     CANCELLED = "cancelled"
7
8 class SimulationTask(BaseModel):
9     task_id: str

```

```

10     status: TaskStatus
11     progress: float = 0.0
12     created_at: datetime
13     started_at: Optional[datetime] = None
14     completed_at: Optional[datetime] = None
15     error_message: Optional[str] = None
16     config: SimulationConfig

```

Listing 8: TaskStatus Pydantic 模型

7. 前端实现详解

7.1. Vue 3 应用架构

7.1.1. 组件结构

前端采用 Vue 3 Composition API 构建，主要组件包括：

```

1 frontend/src/
2   components/      # Vue 组件
3     Scene3D.vue    # 3D场景渲染组件
4     ControlPanel.vue # 参数配置面板
5     WaypointList.vue # 航点列表组件
6     LogConsole.vue  # 日志控制台
7     ModelManager.vue # 模型管理组件
8   stores/          # Pinia状态管理
9     scene.ts        # 场景状态
10    waypoints.ts     # 航点状态
11    simulation.ts     # 仿真状态
12  composables/      # 组合式函数
13    useThreeScene.ts # Three.js场景管理
14    useWaypoints.ts  # 航点交互逻辑
15    useHeliosAPI.ts  # API调用封装

```

Listing 9: 前端组件结构

7.1.2. 3D 场景组件实现

```

1 // components/Scene3D.vue
2 import { ref, onMounted, onBeforeUnmount } from 'vue'
3 import * as THREE from 'three'
4 import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls'
5 import { OBJLoader, GLTFLoader, STLLoader } from 'three/examples/jsm/loaders'
6
7 export default {
8   setup() {
9     const container = ref(null)
10    const scene = ref(null)
11    const camera = ref(null)
12    const renderer = ref(null)
13    const controls = ref(null)
14
15    // 初始化3D场景
16    const initScene = () => {

```



```

17 // 创建场景
18 scene.value = new THREE.Scene()
19 scene.value.background = new THREE.Color(0xf0f0f0)
20
21 // 设置相机
22 camera.value = new THREE.PerspectiveCamera(
23     75,
24     container.value.clientWidth / container.value.clientHeight,
25     0.1,
26     1000
27 )
28 camera.value.position.set(50, 50, 50)
29
30 // 创建渲染器
31 renderer.value = new THREE.WebGLRenderer({ antialias: true })
32 renderer.value.setSize(
33     container.value.clientWidth,
34     container.value.clientHeight
35 )
36 container.value.appendChild(renderer.value.domElement)
37
38 // 添加轨道控制
39 controls.value = new OrbitControls(camera.value, renderer.value.domElement)
40 controls.value.enableDamping = true
41
42 // 添加灯光
43 const ambientLight = new THREE.AmbientLight(0xffffff, 0.6)
44 scene.value.add(ambientLight)
45
46 const directionalLight = new THREE.DirectionalLight(0xffffff, 0.4)
47 directionalLight.position.set(100, 100, 100)
48 scene.value.add(directionalLight)
49
50 // 添加地面
51 const groundGeometry = new THREE.PlaneGeometry(1000, 1000)
52 const groundMaterial = new THREE.MeshLambertMaterial({
53     color: 0xcccccc,
54     side: THREE.DoubleSide
55 })
56 const ground = new THREE.Mesh(groundGeometry, groundMaterial)
57 ground.rotation.x = -Math.PI / 2
58 scene.value.add(ground)
59
60 // 开始渲染循环
61 animate()
62 }
63
64 // 动画循环
65 const animate = () => {
66     requestAnimationFrame(animate)
67     controls.value.update()
68     renderer.value.render(scene.value, camera.value)
69 }
70

```

```

71 // 处理窗口大小变化
72 const handleResize = () => {
73   camera.value.aspect = container.value.clientWidth / container.value.clientHeight
74   camera.value.updateProjectionMatrix()
75   renderer.value.setSize(
76     container.value.clientWidth,
77     container.value.clientHeight
78   )
79 }
80
81 onMounted(() => {
82   initScene()
83   window.addEventListener('resize', handleResize)
84 })
85
86 onBeforeUnmount(() => {
87   window.removeEventListener('resize', handleResize)
88 })
89
90 return {
91   container,
92   addWaypoint,
93   clearWaypoints
94 }
95 }
96 }

```

Listing 10: Scene3D.vue 组件核心代码

7.2. Three.js 集成与 3D 交互

7.2.1. 模型加载系统

```

1 // composables/useModelLoader.ts
2 import { ref } from 'vue'
3 import * as THREE from 'three'
4 import { OBJLoader } from 'three/examples/jsm/loaders/OBJLoader'
5 import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader'
6 import { STLLoader } from 'three/examples/jsm/loaders/STLLoader'
7
8 export function useModelLoader() {
9   const models = ref<Map<string, THREE.Object3D>>(new Map())
10   const loading = ref(false)
11   const error = ref<string | null>(null)
12
13   const loadModel = async (url: string, name: string, format: string) => {
14     loading.value = true
15     error.value = null
16
17     try {
18       let loader: any
19       let object: THREE.Object3D
20
21       switch (format.toLowerCase()) {

```

```

22     case 'obj':
23         loader = new OBJLoader()
24         object = await loader.loadAsync(url)
25         break
26     case 'gltf':
27     case 'glb':
28         loader = new GLTFLoader()
29         const gltf = await loader.loadAsync(url)
30         object = gltf.scene
31         break
32     case 'stl':
33         loader = new STLLoader()
34         const geometry = await loader.loadAsync(url)
35         const material = new THREE.MeshStandardMaterial({
36             color: 0x88ccff,
37             wireframe: false
38         })
39         object = new THREE.Mesh(geometry, material)
40         break
41     default:
42         throw new Error(`Unsupported format: ${format}`)
43 }
44
45 // 自动调整模型朝向 (Y-up转Z-up)
46 const bbox = new THREE.Box3().setFromObject(object)
47 const center = bbox.getCenter(new THREE.Vector3())
48 const size = bbox.getSize(new THREE.Vector3())
49
50 // 计算模型的主朝向
51 const maxAxis = size.x > size.y ? 'x' : 'y'
52 const rotationAngle = maxAxis === 'x' ? Math.PI / 2 : 0
53
54 // 旋转模型
55 object.rotation.z = rotationAngle
56 object.position.copy(center.multiplyScalar(-1))
57
58 // 计算包围盒
59 const worldBbox = new THREE.Box3().setFromObject(object)
60 const worldCenter = worldBbox.getCenter(new THREE.Vector3())
61 const worldSize = worldBbox.getSize(new THREE.Vector3())
62
63 models.value.set(name, object)
64
65 return {
66     object,
67     bbox: {
68         min: worldBbox.min,
69         max: worldBbox.max,
70         center: worldCenter,
71         size: worldSize
72     }
73 }
74 } catch (err) {
75     error.value = err instanceof Error ? err.message : 'Failed to load model'

```

```

76     throw err
77   } finally {
78     loading.value = false
79   }
80 }
81
82 return {
83   models,
84   loading,
85   error,
86   loadModel
87 }
88 }

```

Listing 11: 模型加载组合式函数

7.2.2. 航点交互系统

```

1 // composables/useWaypoints.ts
2 import { ref, computed } from 'vue'
3 import * as THREE from 'three'
4
5 interface Waypoint {
6   id: number
7   position: THREE.Vector3
8   mesh: THREE.Mesh
9 }
10
11 export function useWaypoints() {
12   const waypoints = ref<Waypoint[]>([])
13   const selectedWaypoint = ref<Waypoint | null>(null)
14   const isDragging = ref(false)
15
16   // 添加航点
17   const addWaypoint = (position: THREE.Vector3) => {
18     const id = Date.now()
19     const geometry = new THREE.SphereGeometry(2, 32, 16)
20     const material = new THREE.MeshLambertMaterial({
21       color: 0xff4444,
22       transparent: true,
23       opacity: 0.8
24     })
25
26     const mesh = new THREE.Mesh(geometry, material)
27     mesh.position.copy(position)
28     mesh.position.z = 0 // 贴地
29     mesh.userData = { id }
30
31     const waypoint: Waypoint = {
32       id,
33       position,
34       mesh
35     }
36

```

```

37     waypoints.value.push(waypoint)
38     updateConnectionLines()
39
40     return waypoint
41 }
42
43 // 删除航点
44 const removeWaypoint = (id: number) => {
45     const index = waypoints.value.findIndex(wp => wp.id === id)
46     if (index !== -1) {
47         const waypoint = waypoints.value[index]
48         waypoint.mesh.removeFromParent()
49         waypoints.value.splice(index, 1)
50         updateConnectionLines()
51     }
52 }
53
54 // 更新航点位置
55 const updateWaypointPosition = (id: number, position: THREE.Vector3) => {
56     const waypoint = waypoints.value.find(wp => wp.id === id)
57     if (waypoint) {
58         waypoint.position.copy(position)
59         waypoint.mesh.position.copy(position)
60         waypoint.mesh.position.z = 0
61         updateConnectionLines()
62     }
63 }
64
65 // 更新连接线
66 const updateConnectionLines = () => {
67     // 移除旧的连接线
68     const oldLines = scene.value.getObjectByName('waypoint-lines')
69     if (oldLines) {
70         scene.value.remove(oldLines)
71     }
72
73     // 创建新的连接线
74     if (waypoints.value.length > 1) {
75         const points = waypoints.value.map(wp => wp.position.clone())
76         const geometry = new THREE.BufferGeometry().setFromPoints(points)
77         const material = new THREE.LineBasicMaterial({
78             color: 0xffffff,
79             linewidth: 2
80         })
81
82         const line = new THREE.Line(geometry, material)
83         line.name = 'waypoint-lines'
84         scene.value.add(line)
85     }
86 }
87
88 // 拖拽处理
89 const handleMouseDown = (event: MouseEvent) => {
90     // 鼠标射线检测

```

```

91     const raycaster = new THREE.Raycaster()
92     const mouse = new THREE.Vector2()
93
94     mouse.x = (event.clientX / window.innerWidth) * 2 - 1
95     mouse.y = -(event.clientY / window.innerHeight) * 2 + 1
96
97     raycaster.setFromCamera(mouse, camera.value)
98
99     const intersects = raycaster.intersectObjects(
100         waypoints.value.map(wp => wp.mesh)
101     )
102
103     if (intersects.length > 0) {
104         selectedWaypoint.value = waypoints.value.find(
105             wp => wp.mesh === intersects[0].object
106         ) || null
107         isDragging.value = true
108         controls.value.enabled = false
109     }
110 }
111
112 const handleMouseMove = (event: MouseEvent) => {
113     if (isDragging.value && selectedWaypoint.value) {
114         // 计算与地面的交点
115         const raycaster = new THREE.Raycaster()
116         const mouse = new THREE.Vector2()
117
118         mouse.x = (event.clientX / window.innerWidth) * 2 - 1
119         mouse.y = -(event.clientY / window.innerHeight) * 2 + 1
120
121         raycaster.setFromCamera(mouse, camera.value)
122
123         // 创建地面平面
124         const plane = new THREE.Plane(new THREE.Vector3(0, 0, 1), 0)
125         const intersectPoint = new THREE.Vector3()
126
127         raycaster.ray.intersectPlane(plane, intersectPoint)
128
129         // 更新航点位置
130         updateWaypointPosition(selectedWaypoint.value.id, intersectPoint)
131     }
132 }
133
134 const handleMouseUp = () => {
135     isDragging.value = false
136     selectedWaypoint.value = null
137     controls.value.enabled = true
138 }
139
140 return {
141     waypoints: computed(() => waypoints.value),
142     addWaypoint,
143     removeWaypoint,
144     updateWaypointPosition,

```

```

145     handleMouseDown,
146     handleMouseMove,
147     handleMouseUp
148 }
149 }

```

Listing 12: 航点交互组合式函数

8. 后端实现详解

8.1. FastAPI 应用架构

8.1.1. 主应用配置

```

1 # app/main.py
2 from fastapi import FastAPI, HTTPException
3 from fastapi.middleware.cors import CORSMiddleware
4 from fastapi.staticfiles import StaticFiles
5 import uvicorn
6 import os
7 from .config import settings
8 from .api import router as api_router
9 from .api.websocket import websocket_router
10
11 app = FastAPI(
12     title="FeHALS_API",
13     description="FeHALS 航路规划与激光仿真系统API",
14     version="1.0.0",
15     docs_url="/docs",
16     redoc_url="/redoc"
17 )
18
19 # 配置 CORS
20 app.add_middleware(
21     CORSMiddleware,
22     allow_origins=settings.CORS_ORIGINS,
23     allow_credentials=True,
24     allow_methods=["*"],
25     allow_headers=["*"],
26 )
27
28 # 挂载静态文件
29 app.mount("/static", StaticFiles(directory="static"), name="static")
30
31 # 注册路由
32 app.include_router(api_router, prefix="/api")
33 app.include_router(websocket_router, prefix="/ws")
34
35 @app.on_event("startup")
36 async def startup_event():
37     """应用启动时的初始化操作"""
38     # 检查HELIOS++环境
39     from ..services.helios_service import check_helios_environment
40     await check_helios_environment()

```

```

41
42 @app.get("/")
43 async def root():
44     """API 根路径"""
45     return {
46         "message": "FeHALS_API",
47         "version": "1.0.0",
48         "docs_url": "/docs",
49         "status": "running"
50     }
51
52 if __name__ == "__main__":
53     uvicorn.run(
54         "app.main:app",
55         host=settings.HOST,
56         port=settings.PORT,
57         reload=True
58     )

```

Listing 13: main.py - FastAPI 应用入口

8.1.2. 配置管理系统

```

1 # app/config.py
2 from pydantic_settings import BaseSettings
3 from typing import List
4 import os
5
6 class Settings(BaseSettings):
7     # 服务器配置
8     HOST: str = "0.0.0.0"
9     PORT: int = 8000
10
11     # CORS 配置
12     CORS_ORIGINS: List[str] = ["http://localhost:3000", "http://127.0.0.1:3000"]
13
14     # HELIOS++ 路径配置
15     HELIOS_PATH: str = "/usr/local/bin/helios++"
16     HELIOS_REPO: str = "/opt/helios++/repo"
17     HELIOS_ASSETS: str = "/opt/helios++/assets"
18
19     # 仿真配置
20     SIMULATION_TIMEOUT: int = 300 # 秒
21     MAX_POINTCLOUD_SIZE: int = 10000000 # 点云最大点数
22     DEFAULT_OUTPUT_FORMAT: str = "LAS"
23
24     # 文件上传配置
25     UPLOAD_DIR: str = "uploads"
26     MAX_FILE_SIZE: int = 100 * 1024 * 1024 # 100MB
27     ALLOWED_EXTENSIONS: List[str] = [".obj", ".gltf", ".glb", ".stl"]
28
29     # 缓存配置
30     CACHE_DIR: str = "cache"
31     CACHE_TTL: int = 3600 # 1小时

```



```

32
33     class Config:
34         env_file = ".env"
35         case_sensitive = True
36
37 # 全局配置实例
38 settings = Settings()

```

Listing 14: config.py - 应用配置

8.2. API 路由实现

8.2.1. REST 路由

```

1 # api/routes.py
2 from fastapi import APIRouter, HTTPException, UploadFile, File, Depends
3 from typing import List
4 from ...services import (
5     model_service,
6     trajectory_service,
7     simulation_service,
8     result_service
9 )
10 from ...models.schemas import *
11
12 router = APIRouter()
13
14 @router.get("/models")
15 async def get_models():
16     """获取所有模型列表"""
17     return await model_service.get_all_models()
18
19 @router.post("/models/upload")
20 async def upload_model(
21     file: UploadFile = File(...),
22     name: str = None,
23     description: str = None
24 ):
25     """上传3D模型文件"""
26     # 验证文件类型
27     if not file.filename.lower().endswith(tuple(ALLOWED_EXTENSIONS)):
28         raise HTTPException(
29             status_code=400,
30             detail=f"文件类型不支持。支持的格式:{','.join(ALLOWED_EXTENSIONS)}"
31         )
32
33     # 验证文件大小
34     if file.size > MAX_FILE_SIZE:
35         raise HTTPException(
36             status_code=400,
37             detail=f"文件大小超过限制 ({MAX_FILE_SIZE//1024//1024}MB) "
38         )
39
40     try:

```

```

41     model = await model_service.upload_model(
42         file=file,
43         name=name or file.filename,
44         description=description
45     )
46     return {"success": True, "data": model}
47 except Exception as e:
48     raise HTTPException(status_code=500, detail=str(e))
49
50 @router.delete("/models/{model_id}")
51 async def delete_model(model_id: str):
52     """删除指定模型"""
53     try:
54         await model_service.delete_model(model_id)
55         return {"success": True}
56     except FileNotFoundError:
57         raise HTTPException(status_code=404, detail="模型不存在")
58     except Exception as e:
59         raise HTTPException(status_code=500, detail=str(e))
60
61 @router.post("/trajectory/generate")
62 async def generate_trajectory(waypoints: WaypointList):
63     """从航点生成航迹文件"""
64     try:
65         trajectory = await trajectory_service.generate_trajectory(waypoints)
66         return {
67             "success": True,
68             "data": trajectory,
69             "download_url": f"/api/trajectories/{trajectory.id}/download"
70         }
71     except Exception as e:
72         raise HTTPException(status_code=500, detail=str(e))
73
74 @router.post("/simulation/run")
75 async def run_simulation(config: SimulationConfig):
76     """启动仿真任务"""
77     try:
78         task = await simulation_service.run_simulation(config)
79         return {
80             "success": True,
81             "data": task,
82             "ws_url": f"ws://{settings.HOST}:{settings.PORT}/ws/simulation/{task.task_id}"
83         }
84     except Exception as e:
85         raise HTTPException(status_code=500, detail=str(e))

```

Listing 15: api/routes.py - REST 路由定义

8.3. 业务服务层

8.3.1. 航迹生成服务

```

1 # services/trajectory_generator.py
2 import csv

```

```

3 import os
4 from typing import List, Dict, Any
5 from ..models.schemas import Waypoint, WaypointList, Trajectory
6
7 class TrajectoryGenerator:
8     @staticmethod
9     def generate_trajectory(waypoints: WaypointList,
10                             flight_height: float = 100.0,
11                             time_interval: float = 1.0) -> Trajectory:
12         """从航点列表生成HELIOS++航迹文件"""
13
14         # 生成航迹数据点
15         trajectory_data = []
16
17         for i, waypoint in enumerate(waypoints.waypoints):
18             # 计算时间戳 (每秒间隔)
19             timestamp = i * time_interval
20
21             # 添加航迹点
22             trajectory_data.append({
23                 'time': timestamp,
24                 'roll': 0.0,
25                 'pitch': 0.0,
26                 'yaw': 0.0,
27                 'x': waypoint.x,
28                 'y': waypoint.y,
29                 'z': flight_height
30             })
31
32         # 生成航迹文件内容 (CSV格式)
33         trajectory_content = "time,roll,pitch,yaw,x,y,z\n"
34         for point in trajectory_data:
35             trajectory_content += (
36                 f"{point['time']},{point['roll']},{point['pitch']},"
37                 f"{point['yaw']},{point['x']},{point['y']},{point['z']}\n"
38             )
39
40         # 创建轨迹对象
41         trajectory = Trajectory(
42             id=f"traj_{os.urandom(4).hex()}",
43             name=waypoints.name,
44             waypoints=waypoints.waypoints,
45             flight_height=flight_height,
46             time_interval=time_interval,
47             data=trajectory_data,
48             content=trajectory_content
49         )
50
51         return trajectory
52
53     @staticmethod
54     def generate_grid_trajectory(
55         bounds: Dict[str, float],
56         line_spacing: float,

```

```

57     flight_height: float = 100.0,
58     time_interval: float = 1.0
59 ) -> Trajectory:
60     """生成弓字形（网格）航迹"""
61
62     # 计算网格点
63     x_min, y_min = bounds['min_x'], bounds['min_y']
64     x_max, y_max = bounds['max_x'], bounds['max_y']
65
66     points = []
67
68     # 生成往返航点
69     y_line = y_min
70     direction = 1 # 1: 从左到右, -1: 从右到左
71
72     while y_line <= y_max:
73         if direction == 1:
74             x_range = range(int(x_min), int(x_max), int(line_spacing))
75         else:
76             x_range = range(int(x_max), int(x_min), -int(line_spacing))
77
78         for x in x_range:
79             points.append({
80                 'x': float(x),
81                 'y': float(y_line),
82                 'z': flight_height
83             })
84
85         # 下一行
86         y_line += line_spacing
87         direction *= -1
88
89     # 创建航点列表
90     waypoint_list = WaypointList(
91         waypoints=[
92             Waypoint(
93                 id=i+1,
94                 x=p['x'],
95                 y=p['y'],
96                 z=p['z']
97             ) for i, p in enumerate(points)
98         ],
99         name="grid_trajectory"
100     )
101
102     # 生成航迹
103     return TrajectoryGenerator.generate_trajectory(
104         waypoint_list,
105         flight_height,
106         time_interval
107     )

```

Listing 16: services/trajectory_generator.py - 航迹生成

8.3.2. 点云处理服务

```
1 # services/pointcloud_parser.py
2 import laspy
3 import numpy as np
4 import os
5 from typing import Dict, Any, List, Optional
6 from ..models.schemas import PointCloudStats, PointCloudData
7
8 class PointCloudParser:
9     @staticmethod
10     def parse_las_laz(file_path: str) -> Dict[str, Any]:
11         """解析LAS/LAZ格式点云"""
12
13         try:
14             with laspy.open(file_path) as las_file:
15                 las = las_file.read()
16
17                 # 提取点云数据
18                 points = {
19                     'x': las.x.astype(np.float32),
20                     'y': las.y.astype(np.float32),
21                     'z': las.z.astype(np.float32)
22                 }
23
24                 # 如果存在强度信息
25                 if hasattr(las, 'intensity'):
26                     points['intensity'] = las.intensity
27
28                 # 计算统计信息
29                 stats = PointCloudStats(
30                     total_points=len(las),
31                     bbox={
32                         'min': {
33                             'x': float(las.header.min[0]),
34                             'y': float(las.header.min[1]),
35                             'z': float(las.header.min[2])
36                         },
37                         'max': {
38                             'x': float(las.header.max[0]),
39                             'y': float(las.header.max[1]),
40                             'z': float(las.header.max[2])
41                         }
42                     },
43                     mean_z=float(np.mean(las.z)),
44                     max_z=float(np.max(las.z)),
45                     min_z=float(np.min(las.z))
46                 )
47
48                 return {
49                     'points': points,
50                     'stats': stats,
51                     'format': 'LAS/LAZ'
52                 }
```

```

53
54     except Exception as e:
55         raise Exception(f"解析LAS/LAZ文件失败:␣{str(e)}")
56
57     @staticmethod
58     def parse_xyz(file_path: str) -> Dict[str, Any]:
59         """解析XYZ文本格式点云"""
60
61         points = {'x': [], 'y': [], 'z': []}
62         bbox = {'min': {'x': float('inf'), 'y': float('inf'), 'z': float('inf')},
63                'max': {'x': float('-inf'), 'y': float('-inf'), 'z': float('-inf')}}
64
65         try:
66             with open(file_path, 'r') as f:
67                 for line_num, line in enumerate(f, 1):
68                     line = line.strip()
69                     if not line or line.startswith('#'):
70                         continue
71
72                     parts = line.split()
73                     if len(parts) < 3:
74                         continue
75
76                     try:
77                         x, y, z = map(float, parts[:3])
78
79                         # 更新点数据
80                         points['x'].append(x)
81                         points['y'].append(y)
82                         points['z'].append(z)
83
84                         # 更新包围盒
85                         bbox['min']['x'] = min(bbox['min']['x'], x)
86                         bbox['min']['y'] = min(bbox['min']['y'], y)
87                         bbox['min']['z'] = min(bbox['min']['z'], z)
88                         bbox['max']['x'] = max(bbox['max']['x'], x)
89                         bbox['max']['y'] = max(bbox['max']['y'], y)
90                         bbox['max']['z'] = max(bbox['max']['z'], z)
91
92                     except ValueError:
93                         continue
94
95     except Exception as e:
96         raise Exception(f"解析XYZ文件失败:␣{str(e)}")
97
98     # 转换为numpy数组
99     points = {
100         'x': np.array(points['x'], dtype=np.float32),
101         'y': np.array(points['y'], dtype=np.float32),
102         'z': np.array(points['z'], dtype=np.float32)
103     }
104
105     # 计算统计信息
106     stats = PointCloudStats(

```

```

107         total_points=len(points['x']),
108         bbox=bbox,
109         mean_z=float(np.mean(points['z'])),
110         max_z=float(np.max(points['z'])),
111         min_z=float(np.min(points['z']))
112     )
113
114     return {
115         'points': points,
116         'stats': stats,
117         'format': 'XYZ'
118     }
119
120     @staticmethod
121     def downsample_points(points: Dict[str, np.ndarray],
122                           target_count: int = 1000000,
123                           method: str = 'random') -> Dict[str, np.ndarray]:
124         """降采样点云数据"""
125
126         current_count = len(points['x'])
127
128         if current_count <= target_count:
129             return points
130
131         if method == 'random':
132             # 随机采样
133             indices = np.random.choice(
134                 current_count,
135                 target_count,
136                 replace=False
137             )
138         elif method == 'grid':
139             # 网格降采样
140             voxel_size = np.cbrt(current_count / target_count)
141
142             # 计算网格索引
143             x_indices = (points['x'] // voxel_size).astype(int)
144             y_indices = (points['y'] // voxel_size).astype(int)
145             z_indices = (points['z'] // voxel_size).astype(int)
146
147             # 去重
148             unique_indices = np.unique(
149                 np.column_stack([x_indices, y_indices, z_indices]),
150                 axis=0
151             )
152
153             if len(unique_indices) > target_count:
154                 # 如果网格点仍然太多，再随机采样
155                 indices = np.random.choice(
156                     len(unique_indices),
157                     target_count,
158                     replace=False
159                 )
160                 indices = unique_indices[indices]

```

```

161         return {
162             'x': points['x'][indices],
163             'y': points['y'][indices],
164             'z': points['z'][indices]
165         }
166     else:
167         indices = np.arange(current_count)[
168             np.lexsort((z_indices, y_indices, x_indices))
169            ][:target_count]
170     else:
171         raise ValueError(f"不支持的降采样方法: {method}")
172
173     return {
174         'x': points['x'][indices],
175         'y': points['y'][indices],
176         'z': points['z'][indices]
177     }
178
179 @staticmethod
180 def export_pointcloud(points: Dict[str, np.ndarray],
181                       output_path: str,
182                       format: str = 'LAS') -> None:
183     """导出点云数据"""
184
185     if format.upper() == 'LAS':
186         # 创建LAS文件
187         header = laspy.LasHeader(point_format=3, version="1.2")
188         header.offsets = [
189             np.min(points['x']),
190             np.min(points['y']),
191             np.min(points['z'])
192         ]
193         header.scales = [0.01, 0.01, 0.01]
194
195         las = laspy.LasData(header)
196         las.x = points['x']
197         las.y = points['y']
198         las.z = points['z']
199
200         if 'intensity' in points:
201             las.intensity = points['intensity']
202
203         las.write(output_path)
204     else:
205         # 导出为XYZ文本
206         with open(output_path, 'w') as f:
207             for i in range(len(points['x'])):
208                 f.write(f"{points['x'][i]}_{points['y'][i]}_{points['z'][i]}\n")

```

Listing 17: services/pointcloud_parser.py - 点云处理

参考文献

- [1] Khronos Group. WebGL specification[EB/OL]. 2011. <https://www.khronos.org/registry/webgl/specs/latest/>.
- [2] COLVIN S. Pydantic: data validation using python type hints[EB/OL]. 2019. <https://docs.pydantic.dev/>.
- [3] FIELDING R T. Architectural styles and the design of network-based software architectures[D/OL]. University of California, Irvine, 2000. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [4] FIELDING R, NOTTINGHAM M, RESCHKE J. HTTP semantics: 9110[R/OL]. RFC Editor, 2022. <https://www.rfc-editor.org/info/rfc9110>. DOI: 10.17487/RFC9110.
- [5] THOMSON M, BENFIELD C. HTTP/2: 9113[R/OL]. RFC Editor, 2022. <https://www.rfc-editor.org/info/rfc9113>. DOI: 10.17487/RFC9113.
- [6] BISHOP M. HTTP/3: 9114[R/OL]. RFC Editor, 2022. <https://www.rfc-editor.org/info/rfc9114>. DOI: 10.17487/RFC9114.
- [7] OpenJS Foundation. Node.js[EB/OL]. 2009. <https://nodejs.org/>.
- [8] MONTAIGU T. laspy: python library for lidar LAS/LAZ IO[EB/OL]. 2017. <https://laspy.readthedocs.io/>.

A. 部署与配置管理

A.1. 环境变量配置

表 8
环境变量配置说明

环境变量	说明	默认值
FEHALS_HOST	服务器监听地址	0.0.0.0
FEHALS_PORT	服务器端口	8000
HELIOS_PATH	HELIOS++ 可执行文件路径	/usr/local/bin/helios++
HELIOS_REPO	HELIOS++ 仓库路径	/opt/helios++/repo
HELIOS_ASSETS	HELIOS++ 资源目录	/opt/helios++/assets
FEHALS_SIM_TIMEOUT	仿真超时时间（秒）	300
FEHALS_CORS_ORIGINS	允许的跨域源	http://localhost:3000
FEHALS_UPLOAD_DIR	文件上传目录	uploads
FEHALS_CACHE_DIR	缓存目录	cache

A.2. Docker 部署方案

```
1 # Dockerfile
2 FROM python:3.9-slim
3
4 WORKDIR /app
5
6 # 安装系统依赖
7 RUN apt-get update && apt-get install -y \
8     gcc \
9     g++ \
10    git \
11    wget \
12    && rm -rf /var/lib/apt/lists/*
13
14 # 安装HELIOS++
15 RUN wget https://github.com/3dgeo-heidelberg/helios_plusplus/releases/download/v1.0.0/helios++_linux_v1
16     .0.0.tar.gz && \
17     tar -xzf helios++_linux_v1.0.0.tar.gz && \
18     mv helios++ /usr/local/bin/
19
20 # 复制requirements文件并安装Python依赖
21 COPY requirements.txt .
22 RUN pip install --no-cache-dir -r requirements.txt
23
24 # 复制应用代码
25 COPY . .
26
27 # 创建必要的目录
28 RUN mkdir -p uploads cache static logs
29
30 # 设置环境变量
31 ENV FEHALS_HOST=0.0.0.0
```

```

31 ENV FEHALS_PORT=8000
32 ENV HELIOS_PATH=/usr/local/bin/helios++
33 ENV HELIOS_REPO=/app/helios++/repo
34 ENV HELIOS_ASSETS=/app/helios++/assets
35
36 # 暴露端口
37 EXPOSE 8000
38
39 # 启动命令
40 CMD ["python", "run.py"]

```

Listing 18: Dockerfile - 容器化部署

A.3. 服务监控与日志

```

1 # 使用 Prometheus + Grafana 监控
2 # prometheus.yml
3 global:
4   scrape_interval: 15s
5
6 scrape_configs:
7   - job_name: 'fehals'
8     static_configs:
9       - targets: ['fehals-backend:8000']
10    metrics_path: '/metrics'
11    scrape_interval: 15s

```

Listing 19: 监控配置示例

B. 开发规范与最佳实践

B.1. 代码规范

- **Python 代码**: 遵循 PEP 8 规范, 使用 black 格式化, flake8 检查
- **Vue 代码**: 使用 ESLint + Prettier, 采用 Composition API
- **Git 提交规范**: 使用 Angular 提交格式 (type(scope): message)
- **分支管理**: master (主分支)、develop (开发分支)、feature/* (功能分支)

B.2. API 设计原则

- **RESTful**: 遵循 REST 设计规范, 使用标准 HTTP 方法
- **状态码**: 使用标准 HTTP 状态码, 如 200、201、400、404、500 等
- **错误处理**: 统一错误格式, 包含错误代码和描述信息
- **文档化**: 所有 API 接口必须有 Swagger/OpenAPI 文档

C. 故障排查指南

表 9

常见问题及解决方案

问题现象	解决方案
HELIOS++ 无法启动	检查 HELIOS_PATH 环境变量，确认可执行文件存在
模型文件上传失败	检查文件格式和大小，确认在允许范围内
仿真任务卡住	检查 HELIOS++ 资源目录，查看任务日志
WebSocket 连接断开	检查 CORS 配置，确认代理设置正确
点云渲染性能差	使用降采样，减少同时渲染的点数